StreamKG-Health: Production-Scale Dynamic Knowledge Graph Embedding for Failure-Rate Prediction in Live Kubernetes and Hadoop Platforms

A new method synthesized from Paper 1 and Paper 3

Full SCI-Style Research Manuscript Draft
Recommended Pair 3: Paper 1 + Paper 3

Author name(s) omitted for review
Affiliation and corresponding-author information to be added

Research integrity note: This manuscript proposes a new unified method. Numerical results attributed to the source studies are clearly identified as prior evidence; no experimental results for the proposed unified model are fabricated.

Abstract

Production big-data platforms produce evolving streams of service interactions and infrastructure state. Large live events, workload spikes, autoscaling, rescheduling, and data-placement changes can alter communication patterns without necessarily causing failure, while subtle under-tested dependencies and resource bottlenecks may create dangerous degradation that conventional threshold systems miss. This paper proposes StreamKG-Health, a production-oriented framework that combines multi-snapshot graph autoencoder embeddings from live interaction graphs with dynamic semantic knowledge graphs for contextual failure-rate prediction. The method is motivated by Paper 1, which uses a GCN-based graph autoencoder to learn minute-level service embeddings and detect under-represented services by comparing load-test and live-event behavior, and Paper 3, which demonstrates that dynamic knowledge-graph context improves anomaly detection in changing microservice environments. StreamKG-Health introduces a dual-view representation: a weighted directed interaction graph captures runtime service communication and traffic intensity, while a dynamic knowledge graph captures deployment, ownership, infrastructure, storage, and dependency semantics. A multi-snapshot GCN-GAE learns scalable structural embeddings; contextual neighborhood features are extracted from the knowledge graph; and a fusion model learns temporal mismatch, degradation, and dependency changes. Multi-task output heads estimate current anomaly, future failure probability, discrete hazard, and platform-level failure rate. The framework is designed for high-scale live environments and can be instantiated for Kubernetes, Hadoop, and mixed cloud-native data stacks. Its evaluation protocol includes event-vs-load-test comparison, synthetic and controlled failure injection, natural incident replay, temporal generalization, calibration, lead-time analysis, and root-cause localization. The proposed approach addresses a gap between production graph-stream anomaly detection and semantically grounded, predictive reliability intelligence.

Keywords: graph autoencoder; dynamic knowledge graph; live graph streams; failure-rate prediction; Kubernetes; Hadoop; microservices; anomaly detection; graph embedding; AIOps

1. Introduction

Modern streaming, media, e-commerce, and data-processing platforms are composed of large numbers of interacting services. During peak events, one user action may traverse many components, and small upstream changes can propagate through hidden dependencies. Capacity tests and gamedays are essential, but they do not always reproduce the service-mix and interaction patterns of real traffic. The result is a reliability gap between what was tested and what actually occurs in production.

Paper 1 addresses this gap at production scale by representing minute-level service interactions as directed weighted graphs and learning node embeddings with a Graph Convolutional Network-based Graph Autoencoder. It compares embeddings between load tests and live events to identify services whose behavior is under-represented or anomalous. The study reports promising synthetic-evaluation precision of 96%, a false-positive rate of 0.08%, and recall of 58% under conservative propagation assumptions. Its main contribution is scalable graph representation learning over live microservice graph streams.

However, an interaction graph alone says that two services communicate; it does not necessarily explain where those services run, which controllers own them, which storage resources they depend on, whether autoscaling changed their placement, or whether infrastructure events altered their behavior. Paper 3 provides this missing semantic context through dynamic knowledge graphs built from resource and network monitoring. It shows that dynamic contextual features can improve anomaly detection when topology changes over time.

StreamKG-Health combines the scale-oriented graph-embedding approach of Paper 1 with the dynamic semantic context of Paper 3. The proposed system maintains two synchronized views. The interaction view captures directed weighted runtime traffic and behavior. The knowledge view captures typed entities and relationships such as workload placement, ownership, storage, service dependency, replication, and control-plane structure. The two views are fused to predict not only current mismatch or anomaly but the probability that a component will fail within future horizons.

This paper contributes a dual-view graph model, a multi-snapshot GCN-GAE for scalable representation learning, context-aware fusion, temporal failure hazard estimation, and graph-grounded explanations. The design explicitly targets live Kubernetes and Hadoop environments where graph size, topology churn, and event volume make fully synchronous deep temporal graph models expensive.

2. Source Solution 1: Multi-Snapshot Graph Embedding for Live Services

Paper 1 models service-to-service traffic as minute-level directed weighted graphs. The graph autoencoder uses a GCN encoder to produce node embeddings and a decoder to reconstruct connectivity. Rather than requiring one long temporal model, the training procedure samples multiple graph snapshots. This provides scalability and makes the learned representation sensitive to structural service behavior.

Anomaly scoring is based on the cosine similarity between embeddings from reference load tests and embeddings from actual event traffic. Services with low similarity are candidates for under-testing, operational anomalies, or hidden dependency changes. The approach is especially attractive for high-scale live systems because it can process independent snapshots and compare learned representations without forcing all history into one recurrent network.

The limitation is semantic sparsity. A service interaction graph may show that Service A called Service B more often, but it does not directly represent the Node on which B runs, a resource shortage affecting that Node, a deployment rollout, or a storage dependency. StreamKG-Health addresses this by pairing the interaction graph with a dynamic knowledge graph.

3. Source Solution 2: Dynamic Contextual Knowledge Graphs

Paper 3 constructs dynamic knowledge-graph snapshots from monitoring data and explicitly represents the changing state of microservices and infrastructure. Contextual relationships allow the detector to recognize dynamic faults that may not appear as extreme service-level anomalies. The source study emphasizes that static-topology assumptions are problematic in environments with autoscaling, rescheduling, and changing deployment state.

The knowledge graph also creates an explanation path. If a workload becomes anomalous after its hosting node is drained or resource constrained, the graph can expose that relationship. StreamKG-Health therefore uses the KG not simply as another feature table but as the semantic reference against which interaction-embedding deviations are interpreted.

Table 1. Motivation for combining Papers 1 and 3

Capability	Paper 1	Paper 3	StreamKG-Health
Production-scale graph streams	Strong	Moderate benchmark scale	Strong design target
Graph autoencoder embeddings	Yes	No	Yes
Dynamic semantic context	Limited	Yes	Yes
Reference-vs-live comparison	Yes	No	Yes
Explicit future failure risk	No	No	Yes
Root-cause explanation	Limited	Explicit KG features	Embedding deviation + KG paths

4. Dual-View Graph Formulation

4.1 Interaction graph

At time t, the interaction graph $I_t = (V_t, E_t^I, W_t)$ is a directed weighted graph. Vertices represent services, workloads, or selected platform components. An edge from u to v indicates observed runtime interaction during the aggregation window, and the weight can combine request count, bytes, latency, error rate, or another normalized traffic measure.

$$I_t = (V_t, E_t^I, W_t) \quad (1)$$

4.2 Semantic dynamic knowledge graph

The contextual view $K_t = (V_t^K, E_t^K, X_t, R)$ contains typed entities and relations. In Kubernetes, nodes can include Cluster, Node, Namespace, Deployment, ReplicaSet, Pod, Container, Service, and Volume. In Hadoop, nodes can include NameNode, DataNode, ResourceManager, NodeManager, Job, Queue, HDFSBlock, and Host. Relations encode placement, management, storage, replication, scheduling, communication, and dependency.

$$K_t = (V_t^K, E_t^K, X_t, R) \quad (2)$$

4.3 Cross-view identity mapping

A mapping mu connects runtime interaction vertices to semantic graph entities. A logical service may map to a Kubernetes Service plus multiple Pods, or a Hadoop job stage may map to several task containers. The mapping can therefore be one-to-one or one-to-many. This alignment allows a suspicious embedding change to be interpreted through the relevant infrastructure and control context.

$$mu: V_t^I \rightarrow 2^{(V_t^K)} \quad (3)$$

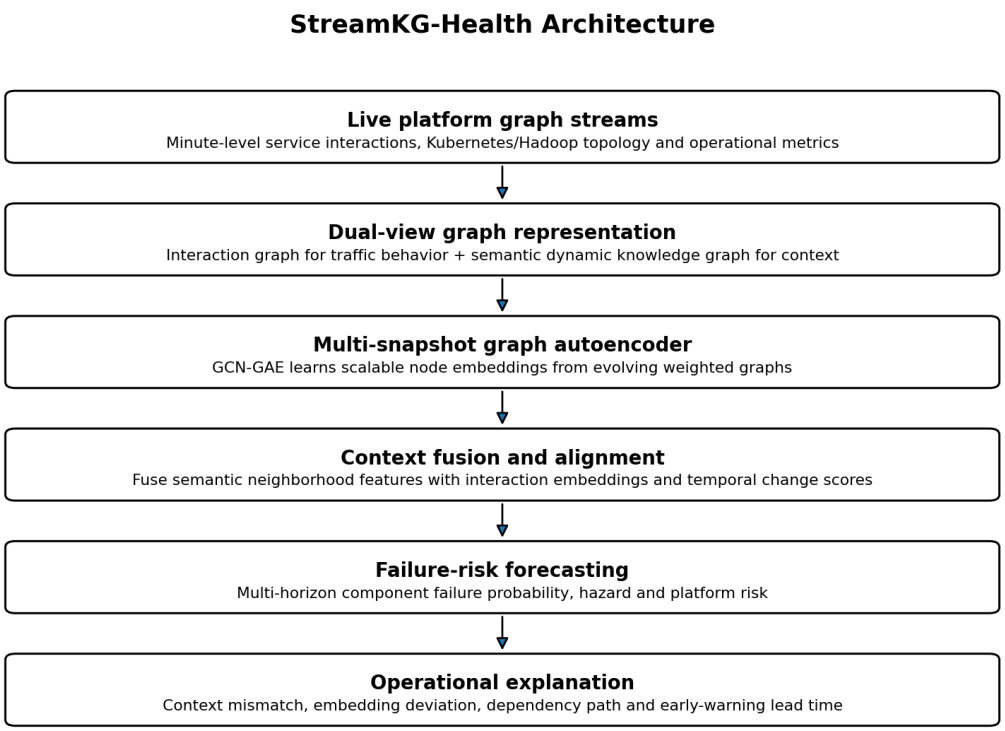

Figure 1. Proposed dual-view StreamKG-Health pipeline.

5. Multi-Snapshot GCN-GAE Representation Learning

The interaction encoder follows the graph-autoencoder principle. For snapshot t, the normalized adjacency and node features are processed by a GCN to produce embedding matrix Z_t. A decoder reconstructs the observed interaction structure. Training across sampled snapshots encourages embeddings that capture recurring structural behavior without requiring one monolithic temporal graph.

For large platforms, node features can include traffic volume, response latency, error rate, in-degree, out-degree, resource usage, and change from baseline. The model can use shared parameters across snapshots and neighborhood sampling for scalability.

$$Z_t = \text{GCN\_Encoder}(X_t, A_t) \quad (4)$$

$$\hat{A}_t = \text{sigmoid}(Z_t Z_t^T) \quad (5)$$

$$L_{GAE} = \sum_t \text{BCE}(A_t, \hat{A}_t) \quad (6)$$

6. Contextual Knowledge-Graph Feature Extraction

The semantic view provides explicit contextual features for each mapped entity. These features can be generated by relation-aware neighborhood aggregation, INK-style explicit walks, or a lightweight KG encoder. Important signals include number of affected neighbors, host-node pressure, controller health, replica availability, storage dependency state, network dependency changes, and recent topology events.

Because context is explicit, the model can distinguish a benign traffic change caused by intentional scaling from a dangerous change accompanied by resource exhaustion or dependency loss. It also improves root-cause explanation because the final risk estimate can be traced to named entities and relations.

7. Context Fusion and Temporal Change Modeling

For each component v, the model combines the interaction embedding z_v^I, semantic context vector c_v^K, embedding deviation from a reference distribution, and recent temporal change statistics. A fusion network can use gated attention so that the model learns when interaction behavior or semantic context is more informative.

Reference behavior can be defined from load tests, previous healthy events, matched workload regimes, or a learned seasonal baseline. Cosine distance measures structural deviation, but the proposed system does not use it as the only anomaly score. Instead, deviation is one input to the failure-risk model.

$$d_v^t = 1 - \text{cosine}(z_v^{\text{reference}}, z_v^t) \quad (7)$$

$$f_v^t = \text{Gate}([z_v^{I,t} \,\|\, c_v^{K,t} \,\|\, d_v^t \,\|\, \text{delta}_v^t]) \quad (8)$$

A lightweight temporal module processes the recent fused states. Because Paper 1 emphasizes snapshot scalability, the default design can use temporal convolution or a GRU over compact fused embeddings rather than a heavy graph-recurrent model over the full graph. This preserves production feasibility while allowing degradation trajectories to influence risk.

$$s_v^t = \text{GRU}(f_v^{(t-L+1)}, \ldots, f_v^t) \quad (9)$$

8. Failure-Risk and Failure-Rate Prediction

The output layer estimates current mismatch or anomaly probability, future failure probability for selected horizons, and discrete hazard. The central target is the probability that a resource or logical service will experience a defined operational failure within Delta, conditioned on being operational at the prediction time.

For live-event platforms, useful horizons may range from minutes to hours. For Hadoop storage and batch workloads, longer horizons may also be relevant. The model can maintain different horizon sets by resource type.

$$\hat{p}_{(v,\Delta)}^t = P(T_v \le t+\Delta \mid T_v > t, I_{[t-L+1:t]}, K_{[t-L+1:t]}) \quad (10)$$

$$P(T_v \le K) = 1 - \prod_{k=1..K}(1 - \hat{q}_{(v,k)}) \quad (11)$$

$$\hat{\lambda}_{(v,\Delta)} = -\ln(1 - \hat{p}_{(v,\Delta)}) / \Delta \quad (12)$$

9. Failure Definition and Label Construction

The model requires a strict distinction between anomalous behavior and failure. A failure label should correspond to measurable service or resource impairment. Kubernetes labels can include NodeNotReady, sustained service unavailability, unsuccessful health checks followed by workload removal, unrecoverable OOMKilled impact, or SLO violation. Hadoop labels can include DataNode loss, unavailable HDFS blocks, repeated job failure, NameNode service interruption, or severe scheduler unavailability.

Historical incident records should be aligned with graph entities and timestamps. Where labels are incomplete, controlled fault injection and deterministic platform events can provide strong labels. Synthetic injection is useful for coverage but should not be the only validation source because synthetic propagation assumptions may differ from real incidents.

10. Explainability and Operational Diagnosis

StreamKG-Health explains risk using four questions: How different is the current interaction embedding from the matched reference? Which semantic context changed at the same time? Which neighboring entities are implicated? Which path connects the suspected cause to the affected service?

An explanation can therefore combine a cosine-deviation score, the top contextual KG features, a ranked dependency path, and the temporal trend of failure probability. This provides more operational context than a single anomaly score. For example, a service may deviate from the load-test embedding because its traffic shifted to a fallback dependency after a storage-side failure. The knowledge graph makes that topology change visible.

11. Experimental Methodology

11.1 Three evaluation regimes

The first regime compares controlled load tests with matched live or replayed event traffic, following the production motivation of Paper 1. The second regime uses controlled fault injection with known start, propagation, and failure times. The third regime uses natural incident replay or historical production traces when available.

A model should not be considered successful merely because it distinguishes load tests from live traffic. The core evaluation is whether the combined representation predicts real failure outcomes early and with calibrated probabilities.

11.2 Kubernetes and Hadoop scenarios

Kubernetes faults should include resource saturation, memory leak, Pod crashes, probe failures, node drain, storage latency, network impairment, replica loss, and dependency failure. Hadoop faults should include DataNode heartbeat loss, disk saturation, under-replication, corrupt blocks, queue overload, NodeManager loss, job retry storms, and control-plane latency.

Workload phases should include baseline traffic, planned scale-up, unexpected service-mix shift, and faulted execution. This is important because the model must separate legitimate high load from degradation.

Table 2. Evaluation questions and metrics

Evaluation question	Metric
Does the model rank failing resources?	AUPRC, AUROC, recall, F1
Are probabilities trustworthy?	Brier score, ECE, reliability diagram
How early is the warning?	Mean and median lead time
Does context improve detection?	Ablation delta with/without KG
Does reference matching help?	Performance across matched vs unmatched load-test baselines
Can the model explain incidents?	Top-k root cause and path accuracy
Can it scale?	Embedding throughput, memory, inference latency

12. Baselines and Ablations

The baseline set should include threshold monitoring, Isolation Forest on flat features, LSTM autoencoder, static GCN anomaly detection, Paper-1-style GCN-GAE cosine scoring, Paper-3-style contextual KG detection, and the full dual-view model. Key ablations remove the knowledge graph, remove the interaction graph, remove reference comparison, remove temporal fusion, or replace native semantics with a kNN similarity graph.

A useful stress test changes the workload composition without injecting a failure. The desired behavior is low failure probability despite significant interaction-embedding movement when the KG explains the change as planned scaling or deployment. Conversely, a subtle embedding deviation accompanied by node pressure and dependency failures should raise risk.

Table 3. Proposed ablations

Ablation	Model change	Interpretation
C1	Interaction graph only	Value of semantic context
C2	KG only	Value of live structural embeddings
C3	No reference comparison	Value of load-test/live mismatch signal
C4	No temporal fusion	Value of degradation trajectory
C5	kNN graph instead of native interaction edges	Value of true operational connectivity
C6	No hazard objective	Value of time-to-failure learning

13. Feasibility Evidence from the Source Studies

The figures in this section are reported by the source studies and are not results of StreamKG-Health. Paper 1 reports synthetic-evaluation precision of 96%, a false-positive rate of 0.08%, and recall of 58% under conservative propagation assumptions. The study also identifies incident-related services and demonstrates early-detection potential in a production Prime Video environment. These observations support the feasibility of scalable graph-embedding comparison for live systems.

Paper 3 reports that context-aware dynamic KG features outperform non-contextual baselines in a difficult benchmark with topology changes. Its best reported online model achieved F1 of 66.7%, and a multi-entity ensemble increased recall to 89.5% at lower precision. These results show that dynamic context can reveal faults that static service-level features miss.

The proposed research hypothesis is that a dual-view representation can preserve the scalability and production sensitivity of graph embeddings while using semantic context to reduce false positives and provide richer failure explanations.

14. Scalability Considerations

The architecture is designed for large graph streams. Independent snapshot encoding can be parallelized. Neighborhood sampling limits GCN cost. Semantic context can be extracted only for entities of interest rather than for every graph vertex. Time windows can be aggregated at multiple resolutions, with fine-grained snapshots near an incident and coarser history elsewhere.

For very large deployments, the platform can be partitioned by namespace, service domain, Hadoop queue, or connected component, with a higher-level meta-graph representing cross-partition dependencies. Embeddings can be cached and updated incrementally when only a small fraction of the graph changes.

15. Discussion

StreamKG-Health addresses a practical tension in AIOps. Production-scale graph streams require efficient representation learning, while meaningful failure diagnosis requires semantic context. Paper 1 is strong on the former and Paper 3 on the latter. The proposed dual-view architecture avoids forcing one graph to perform both roles.

Reference comparison is especially useful when organizations already conduct capacity tests or gamedays. However, the reference must be matched carefully. A service can appear anomalous simply because the workload mix differs. The knowledge graph provides a mechanism to explain such differences through scaling, deployment, placement, or dependency changes.

The method also broadens the concept of failure rate. Rather than calculating one historical average for the entire platform, the system predicts a conditional near-term failure rate for each component and aggregates it by criticality. This is more actionable because risk can change rapidly during a live event.

16. Limitations and Threats to Validity

The proposed dual-view model has not yet been experimentally validated. The availability of matched load-test and live-event data may be limited outside large organizations. Synthetic anomalies can overstate performance if injection rules are too simple. Service identity may also change across deployments, complicating embedding comparison.

The graph autoencoder reconstruction objective can overemphasize frequent edges and high-degree services. Appropriate weighting, negative sampling, and degree normalization are required. Dynamic KG extraction may introduce monitoring overhead, and explanations based on embedding deviation are not inherently causal. Finally, cross-platform application to Hadoop requires careful ontology design and platform-specific failure definitions.

17. Conclusion

This paper proposed StreamKG-Health, a production-oriented dual-view framework for failure-rate prediction in live Kubernetes and Hadoop platforms. The method combines the multi-snapshot GCN-GAE service embeddings and reference-vs-live comparison of Paper 1 with the dynamic semantic knowledge graph and contextual anomaly reasoning of Paper 3.

The resulting architecture learns scalable interaction embeddings, fuses them with explicit infrastructure and dependency context, models recent temporal changes, and predicts calibrated multi-horizon failure probabilities and hazard. It also explains risk through embedding mismatch, contextual features, and graph paths. This synthesis is particularly suitable for platforms that already perform load tests yet need stronger assurance that test behavior matches real production conditions and that emerging deviations are interpreted in their true operational context.

References

[1] S. Madabhushi, P. Vyas, S. Vaidyanathan, M. Kurup, E. Nash, and Y. Silyutin, "From Load Tests to Live Streams: Graph Embedding-Based Anomaly Detection in Microservice Architectures," arXiv:2604.06448, 2026.

[2] P. Moens, B. Steenwinckel, F. Ongenae, B. Volckaert, and S. Van Hoecke, "Toward Context-Aware Anomaly Detection for AIOps in Microservices Using Dynamic Knowledge Graphs," IEEE Transactions on Network and Service Management, vol. 23, pp. 1970-1988, 2026, doi: 10.1109/TNSM.2026.3652304.

[3] T. N. Kipf and M. Welling, "Variational Graph Auto-Encoders," arXiv:1611.07308, 2016.

[4] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," International Conference on Learning Representations, 2017.

[5] P. Velickovic et al., "Graph Attention Networks," International Conference on Learning Representations, 2018.

[6] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-World Challenges and Research Innovations," IEEE/ACM ICSE Companion, pp. 4-5, 2019.

[7] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," ACM CCS, pp. 1285-1298, 2017.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," IEEE International Conference on Data Mining, pp. 413-422, 2008.

[9] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating Explanations for Graph Neural Networks," Advances in Neural Information Processing Systems, vol. 32, 2019.

[10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," Advances in Neural Information Processing Systems, vol. 30, 2017.

[11] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[12] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," International Conference on Learning Representations, 2015.

[13] J. Kosinska and M. Tobiasz, "Detection of Cluster Anomalies with Machine Learning Techniques," IEEE Access, vol. 10, pp. 110742-110753, 2022.

[14] M. A. Rodriguez and P. Neubauer, "The Graph Traversal Pattern," arXiv:1004.1001, 2010.

[15] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.